CPython: Source Code to Bytecode

Study Notes: the Compiler subsystem (compile.c and assemble.c)

Key Idea

The actual compilation from source text into bytecode is handled by the **Compiler subsystem**. It is driven by `Python/compile.c` working alongside `Python/assemble.c`.

The Pipeline (Where Translation Happens)

Step	Stage	What happens
1	**AST passed in**	The Abstract Syntax Tree is passed into Python/compile.c.
2	**Compiler walks the tree**	compile.c traverses the tree, analyzes control flow (loops, if-statements) and generates low-level intermediate pseudo-instructions.
3	**Assembler packages output**	Python/assemble.c takes the pseudo-instructions and packages them into the final executable PyCodeObject (the bytecode).

Flow at a Glance

```
Source text -> Parser -> AST -> Symbol table (scopes)
                              |
                              v
                  Python/compile.c  (pseudo-instructions)
                              |
                              v
                  Python/assemble.c (PyCodeObject = bytecode)
```

Role of Each Component

- **Parser and symbol table:** prepare the structural map and scopes; they do not generate bytecode.
- **Python/compile.c:** walks the AST, analyzes control flow, emits intermediate pseudo-instructions.
- **Python/assemble.c:** the assembler; packs pseudo-instructions into the final PyCodeObject.
- **PyCodeObject:** the executable bytecode container.

Quick Revision Summary

The parser and symbol table build the structural map and scopes. `Python/compile.c` and `Python/assemble.c` are the exact parts of CPython that perform the translation from text-derived trees into compiled bytecode.